

UNIVERSIDAD DE LA CIUDAD DE BUENOS AIRES

LICENCIATURA EN TECNOLOGÍAS INFORMÁTICAS

TRABAJO PRÁCTICO FINAL

PLATAFORMA DE AUDITORÍA DE CÓDIGO BASADA EN INTELIGENCIA ARTIFICIAL PARA LA DETECCIÓN DE VULNERABILIDADES Y REFACTORIZACIÓN SEGURA



ASIGNATURA

Programación de Vanguardia

DOCENTE

Ing. Vázquez Alejandro

CICLO LECTIVO

2026

INTEGRANTES

- Silvina Rebollo
- Natalia Rebollo
- Cecilia Mogro
- Martín Cánepa
- Ricardo Ulloa
- Saúl Zárate

Buenos Aires, Argentina
2026

Contenido

1. Introducción y Justificación	1
2. Diagramas de Arquitectura (Interacción entre Componentes)	2
Flujo de Interacción Secuencial:	2
3. Diseño de Base de Datos (Diagrama Entidad-Relación - DER)	3
4. Especificación de la API (Documentación de Endpoints)	5
Módulo de Inferencia y Análisis (Python)	5
Módulo Web Orquestador (Java / Spring Boot)	6
5. Guía de Despliegue Local y Configuración del Entorno	6
5.1. Requisitos Previos del Sistema	6
5.2. Inicialización de la Base de Datos	7
5.3. Lanzamiento del Microservicio Analítico (Python)	7
5.4. Lanzamiento del Orquestador Web (Java / Spring Boot)	7
5.5. Acceso y Gestión de Roles en la Demora	8
6. Justificación de Elecciones Tecnológicas y de Diseño	9
7. Desafíos Técnicos y Soluciones Implementadas	10
A. Comunicación Inter-Service y Mapeo de JSON Corruptos	10
B. Gestión de Tiempos de Espera (Timeouts) de la API de IA	10
C. Conflicto de Puertos Locales Retenidos (<i>Port Already in Use</i>)	11
8. Lecciones Aprendidas y Conclusiones	11
9. Próximo Paso de Evolución: Arquitectura Contenedorizada con Docker	12
Justificación Técnica de la Evolución:	12

1. Introducción y Justificación

Planteamiento del Problema

En el ámbito académico y profesional de la ingeniería de software, la enseñanza y adopción de prácticas de codificación segura representa un desafío crítico. Los entornos de aprendizaje tradicionales suelen carecer de mecanismos de retroalimentación inmediata, obligando a los estudiantes a depender de revisiones manuales por parte del cuerpo docente o de herramientas de análisis estático de código (SAST) comerciales. Estas últimas, si bien son efectivas en entornos corporativos, presentan interfaces complejas, un alto índice de falsos positivos y carecen de un enfoque pedagógico, limitándose a señalar el error sin explicar el trasfondo técnico ni proponer refactorizaciones didácticas.

Innovación de la Solución Propuesta

La solución desarrollada aborda esta problemática mediante una arquitectura orientada a servicios que fusiona el desarrollo web robusto con Inteligencia Artificial generativa de última generación. La innovación radica en los siguientes pilares:

- **Auditoría Pedagógica Automatizada:** No solo identifica vulnerabilidades bajo estándares internacionales (como el Top 10 de OWASP), sino que reconstruye el código de manera segura y fundamenta técnicamente las modificaciones mediante lenguaje natural adaptado al estudiante.
- **Interoperabilidad Políglota:** El sistema analiza múltiples lenguajes de programación (Java, Python y Kotlin) bajo una misma interfaz unificada.
- **Arquitectura Desacoplada:** El uso de microservicios permite que el núcleo web y el motor de inferencia analítica de IA escalen y evolucionen de forma independiente, garantizando una alta disponibilidad y facilidad de mantenimiento.

2. Diagramas de Arquitectura (Interacción entre Componentes)

El sistema implementa un modelo de arquitectura distribuida compuesto por tres componentes principales interconectados a través de redes locales mediante el protocolo HTTP:



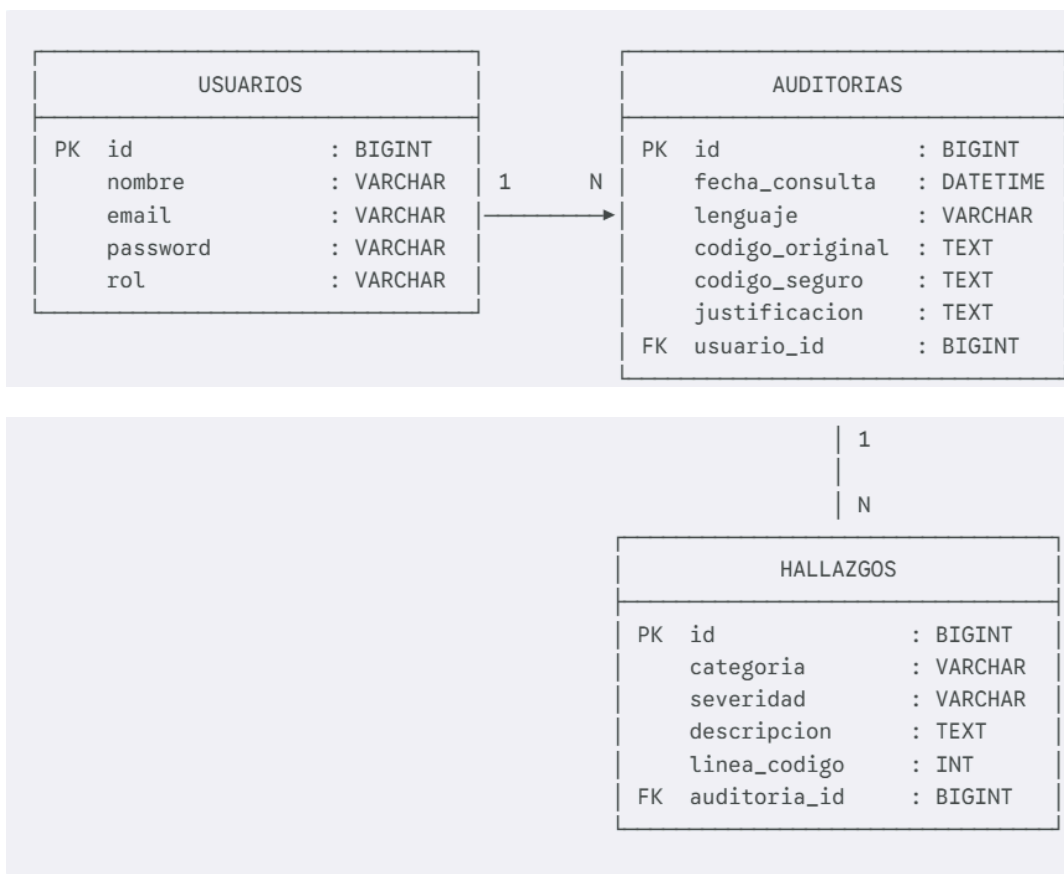
Flujo de Interacción Secuencial:

1. **Capa de Presentación (Frontend):** El usuario inicia sesión de forma segura y accede al panel de control, donde introduce el código fuente a auditar a través de un editor web integrado y selecciona el lenguaje correspondiente (Java, Python o Kotlin).
2. **Capa de Orquestación (Java / Spring Boot):** Recibe la petición del cliente, valida la sesión activa (HttpSession) y encapsula los datos en un Objeto de Transferencia de Datos (RequestDTO). Acto seguido, actúa como cliente HTTP y despacha una solicitud POST hacia el microservicio analítico.

-
3. **Capa de Inferencia (Python / FastAPI):** Recibe el JSON del orquestador, extrae las variables del entorno local de forma segura (API Key de Gemini) y construye un *prompt* de ingeniería avanzada para forzar un rol de "Senior Developer". Se comunica mediante HTTPS con el modelo fundacional de Gemini para procesar el código.
 4. **Retorno y Persistencia:** El microservicio de Python parsea la respuesta de la IA en un formato estructurado y lo devuelve a Spring Boot. El backend de Java persiste de forma inmediata la consulta en la base de datos MySQL para alimentar el historial de auditorías del usuario y, finalmente, renderiza los resultados dinámicamente en la interfaz mediante Thymeleaf.
-

3. Diseño de Base de Datos (Diagrama Entidad-Relación - DER)

La persistencia del microservicio de Java está estructurada de manera relacional mediante el ORM Hibernate. El modelo consta de tres entidades principales mapeadas automáticamente:



Especificación de Relaciones:

- **Usuarios a Auditorías (1:N):** Un usuario registrado en la plataforma puede realizar múltiples consultas de auditoría a lo largo del tiempo. Cada registro de auditoría queda indisolublemente ligado al identificador del auditor que la ejecutó mediante una clave foránea (usuario_id).
- **Auditorías a Hallazgos (1:N):** Una auditoría específica de código puede arrojar múltiples vulnerabilidades o advertencias de seguridad independientes bajo el estándar OWASP. Cada hallazgo apunta a la auditoría madre mediante auditoria_id.

4. Especificación de la API (Documentación de Endpoints)

El intercambio de información entre el ecosistema se rige por especificaciones RESTful estándar. A continuación se detallan los principales endpoints operativos del sistema:

Módulo de Inferencia y Análisis (Python)

- **Endpoint:** POST `http://localhost:8000/api/v1/auditar`
- **Descripción:** Recibe el código plano y el lenguaje de programación; invoca a la API de Gemini y devuelve un análisis de seguridad estructurado.
- **Estructura del Request Body (JSON):**

JSON

```
{  
  "lenguaje": "Java",  
  "codigoFuente": "public void login(String user) { String sql = \"SELECT * FROM  
users WHERE name = '\" + user + '\"\"; }"  
}
```

- Estructura del Response Body (JSON - HTTP 200 OK):

JSON

```
{
  "codigoRefactorizado": "public void login(String user) { String sql =
  \"SELECT * FROM users WHERE name = ?\"; }",
  "explicacionPedagogica": "Se detectó una vulnerabilidad de Inyección SQL. Se
  procedió a parametrizar la consulta utilizando PreparedStatement...",
  "hallazgos": [
    {
      "categoria": "Inyección SQL (OWASP A03:2021)",
      "severidad": "Crítica",
      "descripcion": "Concatenación directa de variables de usuario en sentencias
      SQL dinámicas.",
      "lineaCodigo": 1
    }
  ]
}
```

Módulo Web Orquestador (Java / Spring Boot)

- GET /auth/login y POST /auth/login Control de acceso de usuarios.
- GET /auth/registrar y POST /auth/registrar: Alta de nuevos perfiles en la base de datos (con rol ESTUDIANTE por defecto).
- GET /: Interfaz principal del panel de auditorías. Recupera el historial del usuario desde MySQL.

-
- POST /: Recibe el formulario con el código a auditar, consume la API de Python, persiste los resultados generados en MySQL y refresca la vista del cliente.
-

5. Guía de Despliegue Local y Configuración del Entorno

(Nota: Aquí se integra de manera armónica la guía de despliegue consolidada previamente, asegurando el orden metodológico de la instalación).

5.1. Requisitos Previos del Sistema

El entorno local de desarrollo debe disponer de: **JDK 17**, **Apache Maven 3.8+**, **Python 3.10+** (con soporte para entornos virtuales venv), y un servidor de base de datos **MySQL** activo (configurado mediante XAMPP en sus puertos por defecto).

5.2. Inicialización de la Base de Datos

1. Iniciar el servicio **Apache** y **MySQL** desde el panel de control de XAMPP.
2. Ingresar a `http://localhost/phpmyadmin/` y crear un esquema vacío llamado `auditoria_db`. El ORM de Spring Boot creará de manera automática las tablas (usuarios, auditorias, hallazgos) durante su primer arranque gracias a la directiva `spring.jpa.hibernate.ddl-auto=update`.

5.3. Lanzamiento del Microservicio Analítico (Python)

1. Abrir una terminal y situarse en la raíz del servicio: `cd backend-python`.
2. Crear y activar el entorno virtual aislado:
 - *Windows (PowerShell):* `python -m venv .venv` y luego `.\venv\Scripts\Activate.ps1`.
3. Instalar las dependencias obligatorias: `pip install -r requirements.txt`.

4. **Configuración de Variables de Entorno (Seguridad):** Crear un archivo llamado `.env` en la raíz de esta carpeta e inyectar de manera segura la credencial de desarrollo del modelo fundacional:

Properties

```
GEMINI_API_KEY=tu_clave_privada_de_gemini_aqui
```

5. Ejecutar el script principal para levantar el servicio en el puerto 8000:
`python main.py`.

5.4. Lanzamiento del Orquestador Web (Java / Spring Boot)

1. Abrir una nueva terminal y situarse en el directorio correspondiente: `cd backend-java`.
2. Verificar que las propiedades básicas de conexión en `src/main/resources/application.properties` se encuentren parametrizadas correctamente para un entorno de desarrollo:

Properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/auditoria_db?serverTimezone=UTC
spring.datasource.username=root
spring.datasource.password=
spring.thymeleaf.cache=false
```

3. Limpiar los archivos temporales de compilación y levantar la aplicación web mediante el plugin de Maven:

Bash

```
mvn clean spring-boot:run
```

4. El servidor embebido Tomcat se iniciará con éxito en el puerto **8080**.

5.5. Acceso y Gestión de Roles del Sistema

- Acceder desde el navegador a la dirección `http://localhost:8080/`. El sistema exigirá autenticación previa.
- Al registrar un nuevo usuario desde la interfaz web, el sistema le otorgará de forma automática el rol de **ESTUDIANTE** en cumplimiento de las reglas de seguridad de la aplicación.
- Para realizar pruebas exhaustivas simulando el comportamiento de un docente, se debe alterar el nivel de acceso ejecutando la siguiente consulta directa desde la pestaña SQL de phpMyAdmin:

SQL

```
UPDATE usuarios SET rol = 'PROFESOR' WHERE email =  
'correo_del_docente@unicaba.edu.ar';
```

Al reiniciar sesión con esa cuenta, la barra de navegación del sistema adaptará dinámicamente sus íconos y privilegios reflejando el rol corporativo de **[PROFESOR]**.

6. Justificación de Elecciones Tecnológicas y de Diseño

La arquitectura del sistema fue minuciosamente planificada para mitigar el acoplamiento y asegurar un entorno educativo robusto. A continuación, se fundamentan las decisiones técnicas clave:

- **Java 17 & Spring Boot (Arquitectura Políglota y Orquestador):** Se seleccionó Java para el núcleo del ecosistema debido a su estabilidad institucional, su tipado estático que previene fallos imprevistos en tiempo de ejecución y la madurez de su ecosistema corporativo. Spring Boot gestiona con solvencia la lógica de negocio, el control de sesiones seguras (HttpSession) y la persistencia relacional.
- **Python y FastAPI (Capa de Inferencia de IA):** Las herramientas y SDKs oficiales de Inteligencia Artificial (Google GenAI) están optimizados internacionalmente para Python. Delegar la inferencia de modelos masivos de lenguaje (LLM) a un microservicio independiente en Python previene la sobrecarga de la Máquina Virtual de Java (JVM) y permite un procesamiento asíncrono eficiente.
- **MySQL e Hibernate/JPA (Capa de Persistencia):** Se optó por un motor relacional sólido para estructurar las cuentas de usuarios y mantener una trazabilidad inalterable de los historiales de auditoría. El uso de Hibernate evita la manipulación manual de consultas SQL, previniendo vulnerabilidades y automatizando el esquema.
- **Thymeleaf y Bootstrap 5 (Frontend):** Para optimizar los tiempos de desarrollo y evitar la complejidad de un framework SPA (como React o Angular), se utilizó Thymeleaf. Al renderizar las vistas directamente en el servidor, se centraliza el control de la sesión dentro del contenedor web, impidiendo la manipulación de datos sensibles desde el lado del cliente.

7. Desafíos Técnicos y Soluciones Implementadas

Durante la etapa de integración del ecosistema distribuido, el equipo resolvió tres incidentes críticos de infraestructura:

A. Comunicación Inter-Service y Mapeo de JSON Corruptos

- **El Desafío:** Al enviar fragmentos de código fuente que contenían saltos de línea (`\n`), comillas dobles y tabulaciones, los paquetes HTTP se corrompían frecuentemente, arrojando excepciones de parseo estructural (`JsonParseException`).
- **La Solución:** Se implementó una capa de abstracción estricta mapeando los atributos mediante la librería Jackson en Java y objetos basados en Pydantic en Python. Se forzó la cabecera `Content-Type: application/json; charset=UTF-8`, garantizando el escapado seguro de cadenas complejas a través del protocolo TCP.

B. Gestión de Tiempos de Espera (Timeouts) de la API de IA

- **El Desafío:** La inferencia analítica de la API de Gemini suele demorar entre 2 y 8 segundos dependiendo de la complejidad del algoritmo enviado por el alumno. Este retraso causaba que el servidor embebido Apache Tomcat de Java cortara la comunicación por exceder los tiempos de respuesta por defecto (`Connection timed out`).
- **La Solución:** Se configuró el cliente web de Java (`RestTemplate`) acoplándole una fábrica de conexiones (`HttpComponentsClientHttpRequestFactory`) que establece un tiempo de lectura de socket (*Read Timeout*) extendido a 15 segundos, permitiendo procesar la llamada de forma síncrona sin romper el flujo de la aplicación.

C. Conflicto de Puertos Locales Retenidos (*Port Already in Use*)

- **El Desafío:** Al reiniciar de manera iterativa el entorno con `mvn spring-boot:run`, la consola abortaba indicando que el puerto 8080 ya se

encontraba ocupado. Esto se debía a que los hilos de ejecución de Tomcat quedaban colgados como procesos huérfanos en segundo plano si la terminal se cerraba abruptamente.

- **La Solución:** Se integró un comando de depuración automatizado a nivel de sistema operativo para liberar el socket por la fuerza mediante su Identificador de Proceso (PID) en PowerShell:

PowerShell

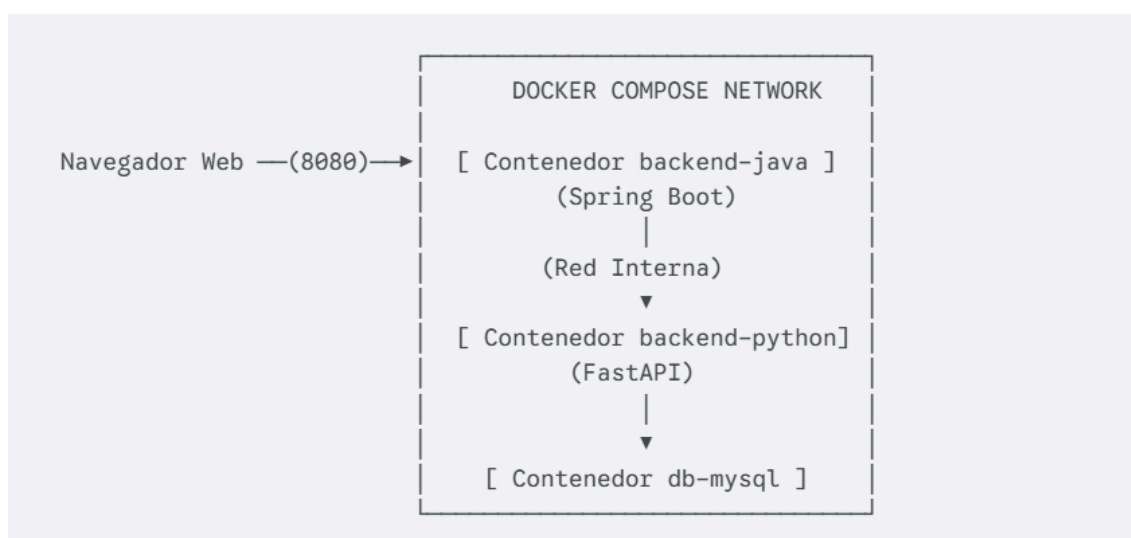
```
Get-Process -Id (Get-NetTCPConnection -LocalPort 8080).OwningProcess |  
Stop-Process -Force
```

8. Lecciones Aprendidas y Conclusiones

- **Resiliencia ante fallos en Microservicios:** Separar la capa de gestión de usuarios (Java) del procesamiento cognitivo (Python) demostró los beneficios del diseño desacoplado. Ante un eventual agotamiento de la cuota de la API de Gemini o una caída del microservicio analítico, la plataforma web principal permanece en línea: el alumno aún puede autenticarse, examinar sus reportes históricos en MySQL y navegar la interfaz con normalidad, mitigando el riesgo de un colapso total del sistema (*Single Point of Failure*).
- **Fricción Operativa Local:** La instalación manual de librerías, la configuración de entornos virtuales (.venv) individuales y la dependencia de que los puertos locales en herramientas como XAMPP no entren en colisión con software de terceros, evidenciaron que el despliegue tradicional añade una alta carga de mantenimiento, demostrando la necesidad de evolucionar hacia estándares industriales de empaquetado inmutable.

9. Próximo Paso de Evolución: Arquitectura Contenedorizada con Docker

Como propuesta formal de mejora tecnológica para la siguiente fase de desarrollo de la institución, se ha planificado la **contenedorización** absoluta del ecosistema empleando **Docker** y **Docker Compose**.



Justificación Técnica de la Evolución:

1. **Garantía Absoluta de Portabilidad:** Al encapsular cada módulo dentro de un contenedor inmutable, se asegura que la plataforma corra de manera idéntica en cualquier computadora, erradicando los conflictos por versiones dispares de runtimes (Java o Python) instalados en las máquinas anfitrionas.
2. **Desmantelamiento de Dependencias Locales (XAMPP):** Docker permite prescindir por completo de herramientas externas de bases de datos. Declarando una imagen oficial de MySQL en el archivo de orquestación, la persistencia se autogestionará en aislamiento absoluto.
3. **Seguridad y Hermeticidad de Red:** Mediante una red privada virtual de tipo bridge, el contenedor de Java se comunicará con el de Python usando alias

de servicio internos (<http://backend-python:8000>). Esto permite cerrar los puertos de Python y MySQL al exterior de la máquina, exponiendo únicamente el puerto web 8080 al usuario final.